-
-
-
-
-
-

-
-
  -
  -
  -
  -
  -

>

- **Memory**
  - Sensory memory
    - Iconic memory (visual)
    - Echoic memory (auditory)
    - Haptic memory (touch)
  - Short-term memory (Working memory)
  - Long-term memory
    - Explicit / Declarative memory (conscious)
      - Episodic memory (life events)
      - Semantic memory (facts, concepts)
    - Implicit / Procedural memory (unconscious; skills)

-
-
-
-
  -
  -
  -

```
Window + Summary
```

•
•
•

•
•
•

①

-
-
-
-
-

-
-
-
-

>


```
->
->
->           /   /
-> Embedding
->      Vector + Metadata
->
```

```
->
->              /     /
->            +   +
->            TopK
->
```

②

•
•
•

•
•
•

-
-
-
-

-
-
-
-

>


```
Agent Runtime
  ├─ Working Memory (window+summary)
  ├─ Memory Orchestrator
  │   ├─ Write Policy
  │   ├─ Retrieval Policy
  │   └─ Forget Policy
  ├─ Vector Store
  ├─ Profile Store
  └─ Audit & Compliance Log
```

```python
class HybridMemoryManager:
    def __init__(self, short_term, vector_store, profile_store):
        self.short_term = short_term
        self.vector_store = vector_store
        self.profile_store = profile_store

    def remember_turn(self, user_id: str, text: str, importance: float):
        self.short_term.append(text)
        if importance >= 0.7:
            self.vector_store.add(text, metadata={"user_id": user_id})

    def recall(self, user_id: str, query: str, k: int = 5):
        profile = self.profile_store.get(user_id)
        docs = self.vector_store.search(query, k=k, filter={"user_id": user_id})
        return {"profile": profile, "memory_docs": docs}
```

-
-
-


```
-> LLM
->          memory_search
->
->
->            memory_write
```

```
#
agent.core_memory_append("                          ")
agent.archival_memory_insert("2026-02-15     A            ...")

hits = agent.archival_memory_search("A            ")
agent.core_memory_replace("            ", "        >     ")
```

③

>

-
-
-

>

		`get_weather`
		`city` `unit`
		`city`

```
tools = [
  {
    "type": "function",
    "function": {
      "name": "get_weather",
      "description": "                              ",
      "parameters": {
        "type": "object",
        "properties": {
          "city": {"type": "string", "description": "              "},
          "unit": {"type": "string", "enum": ["celsius", "fahrenheit"]}
        },
        "required": ["city"]
      }
    }
  }
]
```

```
User Query
 -> LLM decide tool call?
 -> yes: produce tool_call JSON
 -> Executor validate args
 -> Call API / DB / Script
 -> Return tool_result
 -> LLM synthesize final response
```


```python
import asyncio

#        execute_tool                          API/SDK
async def execute_tool(name: str, arguments: dict) -> dict:
    ...

async def run_tool_calls(tool_calls):
    tasks = [
        execute_tool(call["name"], call["arguments"]) for call in tool_calls
    ]
    results = await asyncio.gather(*tasks, return_exceptions=True)
    return results
```

-
-
-
-

④

`search_news` `calc_growth_rate`

-
-
-

>

```
skills/
  finance-analyst/
    SKILL.md
    config.yaml
    prompts/
    tools/
    tests/
```

- [illegible]
- [illegible]

⑤

`weather-mini-skill`

`SKILL.md`

`get_weather`

-
-
-
-

-
-
-

-
-
-


```python
def guard(tool_name, args, user_role):
    validate_json_schema(tool_name, args)
    deny_if_sensitive_fields(args)
    require_approval_if_high_risk(tool_name, user_role)
    return True
```

-
-
-
-

-
-
  -
  -
  -
-

⑥

```
      (    /    )
-> Agent Runtime
-> Memory Layer(       /       )
-> Tool Layer(CRM API/       /    )
-> Approval Layer(       )
-> Audit Layer(       )
```


-
-
-

> [illegible]

- [illegible]
- [illegible]

2

-
-
-


```
Agent            (                  )
     ↓

     ↓
   context
```

-
-
-
-


```
Buffer +                +
```

-
-
-
-
-

```
Recall@K =                    /
```

-
-
-
-

× × ×

- [illegible]
- [illegible]
- [illegible]

- [illegible]
- [illegible]
- [illegible]

- [illegible]

⑦

✅